

Univerzitet u Sarajevu

Elektrotehnički Fakultet

Objektno Orijentisana Analiza i Dizajn

Predmet: Analiza i dizajn sistema — Arhitekturna nadogradnja uvođenjem kreacijskih paterna

Uvod: Na osnovu postojeće specifikacije klasa i funkcionalnih zahtjeva sistema za praćenje aktivnosti djece u predškolskim ustanovama, uočeni su tipični arhitekturni izazovi u segmentima upravljanja korisničkim profilima i generisanja kompleksnih izvještaja. U cilju poboljšanja fleksibilnosti, proširivosti i održivosti koda, predlaže se uvođenje dva kreacijska paterna:

Factory Method i Builder.

1. Factory Method (Fabrička metoda) za hijerarhiju klase Korisnik

Problem u postojećem sistemu: Sistem posjeduje baznu klasu Korisnik i tri izvedene klase: Administrator, Roditelj i Odgajatelj. Prema specifikaciji, atribut uloga unutar klase Korisnik definisan je kao enumeracija (Enum = { RODITELJ, ODGAJATELJ, ADMINISTRATOR }).

Tokom izvršavanja zahtjeva **FZ br. 1 (Registracija korisnika)** i **FZ br. 2 (Prijava korisnika)**, sistem mora dinamički instancirati odgovarajuću podklasu.

Direktno instanciranje unutar klijentskog koda (korištenjem new operatora i switch-case blokova) dovodi do čvrstog vezivanja (tight coupling) i narušava *Open/Closed* princip solid dizajna. Ako se u budućnosti pojavi novi tip korisnika (npr. Direktor ili Pedagog), klijentski kod bi se morao modifikovati na više mjesta.

Rješenje i uloga klase: Uvođenjem fabričke metode, kreiranje objekata se delegira specijalizovanoj klasi:

- KorisnikFactory: Nova klasa koja sadrži fabričku metodu kreirajKorisnika(Uloga uloga). Ona inkapsulira logiku odlučivanja i vraća apstraktni tip Korisnik.
- Korisnik, Administrator, Roditelj, Odgajatelj: Djeluju kao produkti (proizvodi) koje fabrika generiše.

2. Builder (Graditelj) za klasu SazetakAktivnosti

Problem u postojećem sistemu: Klasa SazetakAktivnosti (koja učestvuje u **FZ br. 8:**

Generisanje sedmičnog / mjesečnog sažetka aktivnosti) posjeduje izuzetno veliki broj atributa koji nastaju agregacijom i proračunom podataka iz dnevnih izvještaja. Ti atributi

uključuju: idSazetka, idOdgajatelja, DatumPocetka, DatumKraja, idDjeteta, AvgSpavanja, BrojDolazaka, AgregiraniObroci, OsnovneNapomene i TipSazetka.

Konstrukcija ovakvog objekta putem klasičnog konstruktora stvara problem "anti-patrona dugačkog konstruktora" (Telescoping Constructor). Kod postaje nečitljiv, podložan greškama zbog pogrešnog redoslijeda parametara istog tipa (npr. zamjena datuma ili identifikatora), te nefleksibilan ako su neki od atributa opcionalni (npr. OsnovneNapomene).

Rješenje i uloga klase: Implementacijom Builder obrasca, proces izgradnje kompleksnog objekta se odvija korak-po-korak:

- SazetakAktivnostiBuilder: Nova klasa koja sadrži metode za postepeno postavljanje svakog pojedinačnog atributa (npr. setAvgSpavanja(int avg), setAgregiraniObroci(String obroci)) i metodu build() koja vraća kompletiran objekat.
- Odgajatelj / Sistem za generisanje sažetaka: Koristi graditelja tokom izvršavanja logike zahtjeva **FZ br. 8** kako bi na čitljiv i siguran način konstruisao sažetak nakon sprovedenih proračuna.

Dizajnerski obrazac	Klasa koja se kreira	Funkcionalni zahtjevi (FZ)	Glavna prednost uvođenja
Factory Method	Administrator, Roditelj, Odgajatelj	FZ br. 1, FZ br. 2	Labavo vezivanje, lakše dodavanje novih uloga u sistem bez izmjene klijenskog koda.
Builder	SazetakAktivnosti	FZ br. 8, FZ br. 10	Čitljivost koda, eliminacija predugih konstruktora, kontrolisana korak-po-korak izgradnja kompleksne statistike.

Zaključak: Arhitektonska nadogradnja uvođenjem ovih kreacijskih obrazaca direktno rješava probleme skalabilnosti sistema identifikovane u trenutnoj fazi analize i dizajna, pripremajući backend za stabilnu implementaciju.